

MACHINE VERIFICATION OF THE FIXED $r = 5$ CASE OF ERDŐS PROBLEM 617

RAMAZAN KARA

ABSTRACT. We give an independent machine verification of the fixed $r = 5$ case of an Erdős–Gyárfás balanced-coloring conjecture. In exact form, the Lean theorem states that every five-coloring of the unordered edges of K_{26} has a six-vertex set on which some color is absent. The mathematical source is Robert Sneiderman’s 2026 preprint. The contribution reported here is an independent formal and certificate verification: the coloring semantics, graph reductions, extremal endpoint arguments, residual contradiction, and the final composition are checked in Lean 4.32.0 with a pinned mathlib revision. The remaining finite special-Brooks obstruction is split into 89 exhaustive branches. Each branch has a RUP-only LRAT certificate checked by two external checkers and imported as a propositional theorem into Lean; a kernel-checked semantic and symmetry bridge connects these theorems to the unrestricted graph proposition. A fresh audit checks 89 kernel imports, 89 graph-semantic closures, complete branch coverage, and 192 assumption queries. The final theorem uses only `propext`, `Classical.choice`, and `Quot.sound`.

This verifies only the fixed $r = 5$ case. It does not settle the conjecture for all $r \geq 3$, which remains open.

Scope notice. The machine-verified theorem in this paper is Problem617At(5) only. The unrestricted all- r Erdős Problem 617 conjecture remains open. No claim of independent external review is made.

Attribution and provenance. The original mathematical proof is due to Robert Sneiderman and appears in the Robby955 preprint repository at pinned commit 735c2eeffcb5c8abe92f8a6be04be2fa3c8bb6ab [Sne26]. This paper describes a separate formal-verification project. The formalization is not represented as authorship of the original mathematical proof, nor as external review of that preprint.

1. THE FIXED THEOREM AND ITS BOUNDARY

Erdős and Gyárfás asked whether the following assertion holds for every integer $r \geq 3$: whenever the edges of K_{r^2+1} are colored with r colors, some $r + 1$ vertices induce a complete graph on which at least one color is absent [EG99, Erd99]. The question is listed as Erdős Problem 617 [Blo26].

Date: 24 July 2026.

2020 Mathematics Subject Classification. 05C15, 05C35, 68V20, 03B35.

Key words and phrases. edge coloring, formal verification, Lean, LRAT, Erdős problem.

For finite types V and C , an edge coloring is a total function on the unordered, non-diagonal pairs of vertices,

$$\chi : \binom{V}{2} \longrightarrow C.$$

For $S \subseteq V$, a color $c \in C$ is absent on S when $\chi(e) \neq c$ for every $e \in \binom{S}{2}$. The fixed statement checked in this project is the following.

Theorem 1.1 (Fixed $r = 5$ upper theorem). *For every map*

$$\chi : \binom{[26]}{2} \longrightarrow [5],$$

there are $S \in \binom{[26]}{6}$ and $c \in [5]$ such that c is absent from every edge induced by S .

The corresponding lower bound on 25 vertices is the standard affine construction. Take $V = \mathbb{F}_5^2$. The six parallel classes of affine lines are indexed by the five slopes and the vertical direction. Merge two classes and use the other four as individual colors. Every six points contain two points on a common line in each of the five resulting color classes. Thus every six-set sees every color on K_{25} . Together with Theorem 1.1, this gives the equivalent set-coloring Ramsey identity $R(6; 5, 4) = 26$, as explained in [Sne26].

Remark 1.2 (Why this is not a solution of the full problem). The formal development defines

Problem617At(r) := the upper assertion on K_{r^2+1} with r colors,
and separately defines

$$\text{Problem617} := \forall r, 3 \leq r \longrightarrow \text{Problem617At}(r).$$

The exported theorem has type Problem617At(5), not Problem617. A positive result for one fixed value cannot be generalized by changing a numeral.

2. EXACT LEAN STATEMENT

The formal model uses `SimpleGraph.TopEdgeLabeling`. This prevents two common specification errors: loops are not silently colored, and the two orientations of an edge cannot receive different colors. The central definitions, with notation slightly compressed, are:

$$\begin{aligned} \text{ColorPresent}(\chi, S, c) &: \iff \exists u, v \in S, u \neq v \wedge \chi(\{u, v\}) = c, \\ \text{ColorAbsent}(\chi, S, c) &: \iff \neg \text{ColorPresent}(\chi, S, c), \\ \text{UpperStatement}(V, C, k) &: \iff \forall \chi, \exists S, |S| = k \wedge \\ &\quad \exists c, \text{ColorAbsent}(\chi, S, c). \end{aligned}$$

The fixed theorem is definitionally

$$\text{R5Upper} = \text{UpperStatement}(\text{Fin}(26), \text{Fin}(5), 6),$$

while

$$\text{Problem617At } r = \text{UpperStatement}(\text{Fin}(r^2 + 1), \text{Fin}(r), r + 1).$$

Lean checks by reflexivity that $\text{Problem617At } 5 \leftrightarrow \text{R5Upper}$.

For each color c , the color graph G_c contains exactly the edges colored c . The development proves

$$\text{ColorAbsent}(\chi, S, c) \longleftrightarrow G_c.\text{IsIndepSet}(S),$$

and consequently

$$\text{IsCounterexample}(6, \chi) \longleftrightarrow \forall c, G_c.\text{IndepSetFree}(6).$$

These are ordinary propositions proved from the definitions; the certificate layer does not redefine the original coloring statement.

The final declarations are:

Lean declaration	Type
e058R5SpecialBrooksObstruction	R5SpecialBrooksObstruction
e058NoR5Counterexample	$\neg \exists \chi, \text{IsCounterexample } 6 \ \chi$
e058R5Upper	R5Upper
e058Problem617AtFive	Problem617At 5

3. MATHEMATICAL ARCHITECTURE

3.1. Color graphs and admissibility. Assume for contradiction that every six vertices see all five colors. For a fixed color graph G , every six-set S satisfies

$$1 \leq e(G[S]) \leq 11. \quad (1)$$

The lower bound says that the fixed color occurs. The upper bound follows because each of the other four colors occupies at least one of the 15 edges. In particular,

$$\alpha(G) \leq 5, \quad \omega(G) \leq 5.$$

The formal development calls a graph *admissible* when every six vertices span at most 11 edges. Admissibility is inherited by induced subgraphs.

The five color graphs partition $E(K_{26})$, so a least color has at most $325/5 = 65$ edges. The mathematical proof uses Brooks's theorem and specialized extremal estimates to confine its minimum degree [Bro41, KP05, Sne26]. In the formal development, the imported general theorems are replaced by direct checked specializations. The only finite endpoint isolated as a proposition is

R5SpecialBrooksObstruction :

$$\begin{aligned} \# G \text{ on } \text{Fin}(26) \text{ that is 5-regular and admissible,} \\ G.\text{CliqueFree}(6) \text{ and } G.\text{IndepSetFree}(6). \end{aligned} \quad (2)$$

Sections 4 and 5 explain how (2) is proved in the kernel rather than assumed.

3.2. Independence layers. The non-certificate portion of the formal proof develops three layers of induced-subgraph estimates. We record their roles because they explain the later residual recursion.

- (1) If F is admissible, $\alpha(F) \leq 2$, and $n = |V(F)| \geq 12$, then the complement has maximum degree at most four, giving

$$e(F) \geq \binom{n}{2} - 2n.$$

The order-11 endpoint is proved separately: $e(F) \geq 36$.

- (2) For admissible graphs with $\alpha(F) \leq 3$, the formal proof propagates the first layer and proves exact lower bounds 45, 50, and 54 at orders 16, 17, and 18. The order-10 equality analysis identifies the balanced two-fold blow-up of C_5 and its exact cover structure.
- (3) For admissible graphs with $\alpha(F) \leq 4$, the next propagation gives lower bounds 55, 59, and 62 at orders 21, 22, and 23. Exact order-15 structures at 35 and 36 edges supply the equality cases required by the final argument.

These statements are quantified over arbitrary finite simple graphs with the displayed hypotheses. The small transparent enumerations use kernel `decide`, not `native_decide`; no graph catalog or solver result is an unmentioned premise.

3.3. Equalization and residual contradiction. Conditional on (2), the least-color reduction yields a vertex of degree two, three, or four. The independence-layer bounds exclude degrees two and three. They also force every color graph to have exactly 65 edges. Equality in the neighborhood accounting isolates a K_5 , leaving an induced graph H on 21 vertices with

$$e(H) = 55, \quad \alpha(H) \leq 4, \quad \delta(H) \geq 4,$$

and with inherited admissibility.

The degree-four residual branch peels a second isolated K_5 and uses the order-10 and order-11 endpoints to prove $\delta(H) \geq 5$. The degree-five branch uses the exact order-15 alternatives, common-avoider lemmas, cover transport through the canonical two-fold C_5 blow-up, and the local six-set cap to derive a contradiction. Lean therefore proves

$$\text{R5SpecialBrooksObstruction} \longrightarrow \neg \exists \chi, \text{IsCounterexample}(6, \chi),$$

and composes this implication with the exact coloring semantics.

The proof-module boundary is summarized in Table 1.

TABLE 1. Principal non-certificate formal layers.

Stage	Lean module	Checked role
Semantics	<code>Basic</code>	Exact coloring and color-graph equivalence
Large terminals	<code>AlphaTwoLarge</code> , <code>AlphaPropagation</code>	Direct admissible density bounds
Exact endpoints	<code>AlphaTwoEleven</code> , <code>OrderTenStructure</code>	Orders 10 and 11
Middle layers	<code>AlphaThreeSmall</code> , <code>OrderFifteenStructure</code>	Independence three and exact order 15
Last layer	<code>AlphaFourSmall</code>	Independence four
Equalization	<code>EdgeEqualization</code>	65 edges and first isolated K_5
Residuals	<code>ResidualDegreeFour</code> , <code>ResidualDegreeFive</code>	Final contradiction, conditional on (2)

4. THE 89 CERTIFICATE BRANCHES

4.1. **Exhaustive finite split.** Fix a graph satisfying the four hypotheses of (2). Relabel a selected vertex as 0 and its five neighbors as $1, \dots, 5$. There are 26 canonical isomorphism types for the graph induced by those five neighbors. The remaining 20 vertices are sorted by their five-bit adjacency rows. The difficult neighborhood type, branch 19, is refined by cross patterns and then by a zero-pattern anchor. The complete leaf inventory is:

Family	Leaves	Meaning
E038	19	Neighborhood branches $0, \dots, 18$
E042	17	Direct cross-pattern children of branch 19
E043	47	Residual zero-anchor children: $26 + 16 + 5$
E045	6	Final quotient branches $20, \dots, 25$
Total	89	Pairwise named exhaustive leaves

Each leaf is encoded as a CNF over 325 primary graph-edge variables and deterministic auxiliary variables for cardinality and pattern constraints. The E038, E042, and E043 leaves share an allocation scheme with 3930 variables; the six E045 leaves use a second scheme with 3920. The clauses follow necessarily from regularity of degree five, admissibility, the absence of a six-clique or an independent six-set, and the branch’s canonical unit assignment. Hence unsatisfiability of every leaf excludes the graph family, provided both encoding soundness and branch coverage are proved.

4.2. **RUP-core slicing.** The branch proofs are LRAT traces containing only reverse unit-propagation additions. A deterministic backward slicer starts at the final empty clause, follows every antecedent, retains precisely the reachable original and derived clauses, and renumbers them monotonically. It rejects RAT hints, malformed or repeated identifiers, missing antecedents, comments in the kernel input, and a missing final empty clause.

For every leaf, the reduced CNF/LRAT pair is accepted by:

- (1) the compiled C checker derived from `drat-trim` [WHH14]; and
- (2) a separately implemented Python RUP checker.

Both checkers reject two deterministic corruptions per leaf: a truncated proof and a forged empty clause. This gives four negative checker outcomes per leaf, so the replay ledger contains 89 C checks, 89 Python checks, and 356 negative checks. Separate unit tests reject dependency and renumbering damage. These external checks are defense in depth. They do not replace the Lean import.

TABLE 2. Aggregate original and backward-core certificate inventory.

Artifact family	Bytes	Clauses/additions
Original CNFs (decompressed)	2,709,936,427	43,878,934 clauses
Original LRATs (decompressed)	795,950,803	1,274,831 additions
Reduced CNFs	5,723,342	183,145 clauses
Reduced LRATs	441,983,911	1,274,831 additions
Old-to-new maps	48,467,568	89 maps

Every one of the 1,274,831 retained additions is RUP; the aggregate RAT count is zero. The slicing gain comes chiefly from removing unused original clauses and deletion records while retaining the complete reachable derivation.

4.3. Lean LRAT import. Mathlib’s LRAT machinery constructs a term of type

`Sat.Fmla.proof  $\Gamma$  []`,

which says that every valuation satisfying the retained input formula Γ satisfies the empty clause [CFHH⁺17, The20]. Large proofs are split into dependency-ordered stages to bound elaborator memory. Each stage imports the theorem produced by the previous stage; the last stage exports a named unsatisfiability theorem. The Lean kernel checks all stages. The generated theorem sources and final object files are hashed in per-leaf receipts.

Backward slicing can delete primary input clauses that are unnecessary for a particular refutation. For this reason, a hashed unit manifest is generated from the exact original branch formulas, not inferred from the reduced core. The semantic layer proves that any graph satisfying the branch hypotheses gives a valuation satisfying:

- (1) every retained primary graph clause;
- (2) every retained sequential-counter clause;
- (3) every retained exterior-pattern clause; and
- (4) the exact canonical unit assignment.

Combining this valuation theorem with the imported empty-clause theorem gives one graph-semantic contradiction per leaf.

5. KERNEL-CHECKED SYMMETRY AND SEMANTIC BRIDGE

Certificate checking alone would prove only that 89 propositional formulas are unsatisfiable. E058 therefore includes a separate proof of the route from an arbitrary graph in (2) to one of those formulas.

5.1. Normalization. The development constructs an explicit vertex permutation that fixes vertex 0 and sends its actual five-element neighborhood to $\{1, 2, 3, 4, 5\}$. It proves that graph relabeling preserves degree, admissibility, clique-freeness, and independence-freeness. A second permutation fixes the first six vertices and lexicographically sorts the 20 exterior rows.

5.2. Orbit coverage. Transparent generated witness tables prove that every five-vertex neighborhood graph belongs to one of the 26 canonical orbits. Branch 19 receives a separately checked cross-pattern orbit table. The three remaining zero-anchor parents receive checked column and row actions and witness-child tables. These computations use ordinary reducible definitions and kernel `decide`; their generator output is regenerated byte-for-byte in the fresh audit.

The coverage modules establish:

- E038 leaves $\implies$ branches 0, $\dots$, 18 impossible,
- E042/E043 leaves $\implies$ branch 19 impossible,
- E045 leaves $\implies$ branches 20, $\dots$, 25 impossible.

The final composition is the following checked implication chain:

$$\begin{array}{c}
 89 \text{ kernel LRAT theorems} \\
 \Downarrow \\
 89 \text{ graph-semantic contradictions} \\
 \Downarrow \\
 26 \text{ canonical neighborhood branches impossible} \\
 \Downarrow \\
 \text{R5SpecialBrooksObstruction} \\
 \Downarrow \\
 \text{R5Upper} \iff \text{Problem617At 5.}
 \end{array}$$

No symmetry assumption is inserted as an axiom, and no receipt or Python Boolean is coerced into a theorem.

6. FRESH AUDIT AND VERIFICATION LEVEL

The recorded final release validation is one uninterrupted audit of exact commit `d19a0cf786a0fa714289830f276cf406408ab65b`. The mirror contained the separately compiled C checker but no local Lean build directory when the audit began. The process completed every gate after 25,497.97 seconds. Its receipt, with SHA-256 `789acb2bc829fcba0a5656fabca5a37d9d4fdd2b493858039ad0081cc37032d1`, records zero warnings, zero forbidden source hits, a true `fresh_local_build` field, and a false `resumed_after_core_replay` field.

Earlier development evidence is retained with its original qualifications. A successful clean-start sequence used post-core resume segments, and a later hardened final-source phase explicitly reported itself as a resume while revalidating retained stages, generated sources, coverage modules, and assumptions. A provenance receipt binds that history to its legacy runner and terminal receipt. Two later exact-source attempts that produced no receipt are also retained as infrastructure diagnostics: one was terminated at a command-session boundary, and one lacked the separately built C checker. The uninterrupted exact-commit run removes the resume qualification from the final-source evidence without rewriting this history.

All successful runs pin Lean to `leanprover/lean4:v4.32.0` and `mathlib` to revision `81a5d257c8e410db227a6665ed08f64fea08e997`. The audit then performs the following gates in order:

- (1) regenerate the three transparent orbit-certificate sources and compare them byte-for-byte;
- (2) build the base library and the E058 semantic bridge with zero warnings;
- (3) regenerate all 89 reduced RUP cores from the committed compressed inputs;
- (4) run both external checkers and all 356 corruption tests;
- (5) regenerate and hash all 89 primary unit CNFs;
- (6) re-split every core, regenerate each committed stage source byte-for-byte, and freshly import every staged kernel proof;
- (7) build all 89 graph-semantic closure theorems;
- (8) compile all eight coverage modules;

- (9) query the assumptions of 89 kernel theorems, 89 semantic theorems, and 14 coverage/final theorems; and
- (10) scan every audited source for `sorry`, `admit`, project `axiom`, `unsafe`, `native_decide`, `ofReduceBool`, and `run_tac`; compare source hashes before and after the run.

TABLE 3. Fresh E058 audit ledger.

Gate	Passed items
Reduced RUP cores	89
External C/Python positive checks	89 + 89
Deliberate corruption rejections	356
Lean kernel LRAT imports	89
Lean graph-semantic closures	89
Coverage modules	8
Assumption queries	192
Warnings	0
Forbidden source hits	0

Diagnostic benchmark outcome. The preregistered proof-only import of the unreduced E038 branch-02 CNF/LRAT did not complete within its 300-second cap on the final audit machine. GNU `time` recorded exit status 124, 310.08 user seconds, 7.57 system seconds, 5:19.34 elapsed including termination grace and cleanup, and peak resident memory of 5,924,372 KiB. The exact source, empty Lean output, and resource log are retained. Consequently no raw-versus-core speedup ratio is claimed. The theorem-bearing reduced core was independently regenerated, checked, staged, and accepted by Lean in every successful E058 audit. This failed diagnostic performance gate is not a logical premise of the kernel theorem, but it is reported because it was preregistered.

The assumption audit reports that `Erdos617.e058Problem617AtFive` depends exactly on

`{propext, Classical.choice, Quot.sound}`.

These are standard logical foundations used by `mathlib`. In particular, the dependency list contains no `sorryAx` and no project-defined axiom.

Under the repository’s verification vocabulary, this is a *machine-verified resolution of the fixed $r = 5$ scope*. It is not an externally verified result: the formal development and this preprint have not yet completed independent expert review. It is also not a resolution of the quantified all- r conjecture.

7. DISCLOSURE OF AI ASSISTANCE

OpenAI Codex was used as an AI-assisted research and software engineering tool in this project. It helped draft and revise Lean, Python, shell, documentation, and $\text{\LaTeX}$ material; execute verification workflows; inspect intermediate results; and organize the reproducibility record. The formal and expository output should therefore not be read as unaided human work. Codex is not an author and its textual assertions are not evidence for any

mathematical claim. The internal adversarial audit was likewise Codex-assisted and is not represented as independent human review.

Every machine-verification claim in this paper is instead tied to committed review source, certificates, kernel-checked theorem objects, audit transcripts, and cryptographic hashes that can be replayed independently. Ramazan Kara directed the project, selects what may be published, and assumes responsibility for any published artifact and for this account.

8. TRUST BOUNDARY

Proof premises and reproducibility checks are trusted differently.

Logically trusted. The final theorem is accepted by the Lean 4 kernel. The practical trusted computing base includes the Lean parser, elaborator and kernel executable, the operating system and hardware used to run it, and the standard logical axioms printed above [dMU21]. The pinned mathlib sources supply graph theory infrastructure and the LRAT proof constructor [The20].

Not trusted as theorem oracles. The SAT solvers that originally produced the LRAT traces are not theorem premises. Neither are the Python and C external checkers, the core slicer, the certificate generators, the audit receipts, or hash comparisons. An error in one of these tools can at worst cause it to propose a bad proof term or a mismatched source; the Lean kernel must still accept the resulting term, and the graph-semantic and coverage theorems must still type-check.

Why hashes still matter. Kernel checking establishes the proposition compiled in a particular run. Hashes establish that the reviewed or eventually released sources, compressed certificates, generated units, transcripts, and receipts are the same artifacts described by this paper. They close the distribution and replay boundary, not the logical inference boundary.

Excluded claims. No assertion is made here about the correctness of claimed fixed cases $r = 6, \dots, 9$, except that they are prior work outside this verification. No unsuccessful search, solver agreement, or unrestricted SAT timeout is used as evidence. The full statement $\forall r \geq 3, \text{Problem617At}(r)$ remains open.

9. REPRODUCTION

The local release candidate contains:

- all pinned Lean source files, `lean-toolchain`, `lake-manifest.json`, and the project lakefile;
- the 89 original compressed CNFs and LRAT certificates;
- the core slicer, both external checkers, stage generator, semantic runner, complete fresh-audit runner, and their unit tests;
- the exact unit manifest, fresh receipts, audit source and output, transcripts, resource records, and SHA-256 inventory;
- this complete L^AT_EX source, bibliography, build file, and rendered PDF.

From the repository root, build the small external checker and run the complete E058 unit-test target:

```
make verify-e058-unit
```

Create a source mirror that has no `formal/lean/.lake/build` directory and run:

```
python3 repro/run_e058_special_brooks_kernel_audit.py \
  --project /fresh --lake /lean-4.32.0/bin/lake \
  --log /fresh/e058-audit.log --require-no-local-build
```

The success line begins

```
E058-FRESH-LEAN-AUDIT-PASS cores=89 kernel_imports=89
semantic_closures=89 axiom_queries=192
warnings=0 forbidden_hits=0
```

The retained final audit ran this command without interruption from exact commit `d19a0cf786a0fa714289830f276cf406408ab65b`; the C checker was compiled first, while the local Lean build directory remained absent. Its receipt has SHA-256 `789acb2bc829fcb0a5656fabca5a37d9d4fdd2b493858039ad0081cc37032d1`. The earlier clean-start/resume sequence, hardened replay, receipt-assembly failure, and two exact-source infrastructure failures are retained and explicitly labeled; none is substituted for the uninterrupted receipt. Finally, verify the release inventory with:

```
sha256sum --check \
  artifacts/e058_special_brooks_kernel_bridge/SHA256SUMS
```

The release reproduction guide records the exact mirror command, expected tool hashes, receipt hashes, and required disk and memory envelope.

10. CONCLUSION

The checked declaration

```
Erdos617.e058Problem617AtFive : Problem617At 5
```

establishes that every five-coloring of K_{26} has six vertices omitting a color. The result is an independent formal and certificate verification of the fixed- $r = 5$ theorem whose mathematical source is Sneiderman’s preprint. Its main verification contribution is the end-to-end bridge from 89 LRAT certificates, through exact graph semantics and exhaustive symmetry coverage, to the unconditional original coloring statement.

The quantified Erdős Problem 617 conjecture for every $r \geq 3$ remains open.

REFERENCES

- [Blo26] Thomas F. Bloom. Erdős problem 617. <https://www.erdosproblems.com/617>, 2026. Accessed 24 July 2026.
- [Bro41] R. L. Brooks. On colouring the nodes of a network. *Mathematical Proceedings of the Cambridge Philosophical Society*, 37(2):194–197, 1941.
- [CFHH⁺17] Luis Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Jr., Matt Kaufmann, and Peter Schneider-Kamp. Efficient certified RAT verification. In *Automated Deduction – CADE 26*, volume 10395 of *Lecture Notes in Computer Science*, pages 220–236, 2017.
- [dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, 2021.
- [EG99] Paul Erdős and András Gyárfás. Split and balanced colorings of complete graphs. *Discrete Mathematics*, 200(1–3):79–86, 1999.

- [Erd99] Paul Erdős. A selection of problems and results in combinatorics. *Combinatorics, Probability and Computing*, 8(1–2):1–6, 1999.
- [KP05] Mihyun Kang and Oleg Pikhurko. Maximum K_{r+1} -free graphs which are not r -partite. *Matematychni Studii*, 24(1):12–20, 2005.
- [Sne26] Robert Sneiderman. The five-color case of an erdős-gyárfás balanced-coloring problem. Preprint and source repository, July 2026. Repository commit [735c2eeffc5c8abe92f8a6be04be2fa3c8bb6ab](#).
- [The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.
- [WHH14] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429, 2014.